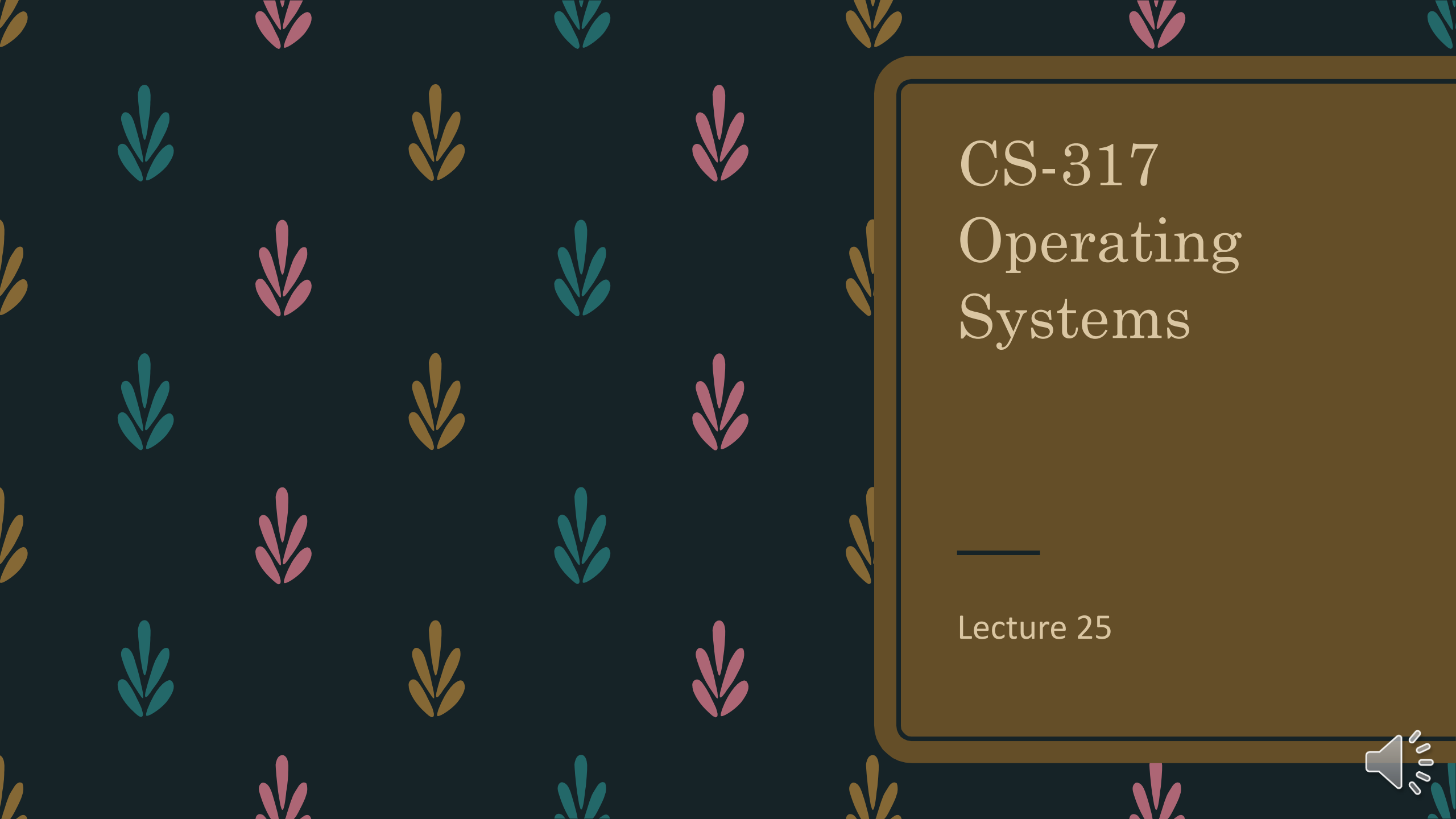




CS-317

Operating Systems

Lecture 25



Deadlocks

Book 2 Chapter 7



From the previous lecture...

We understood what a deadlock is.

We realized that there are two types of resources in a computer system.

We explored the conditions for deadlock.

We listed the approaches to deal with deadlock.

We explored deadlock prevention.



Deadlock Avoidance

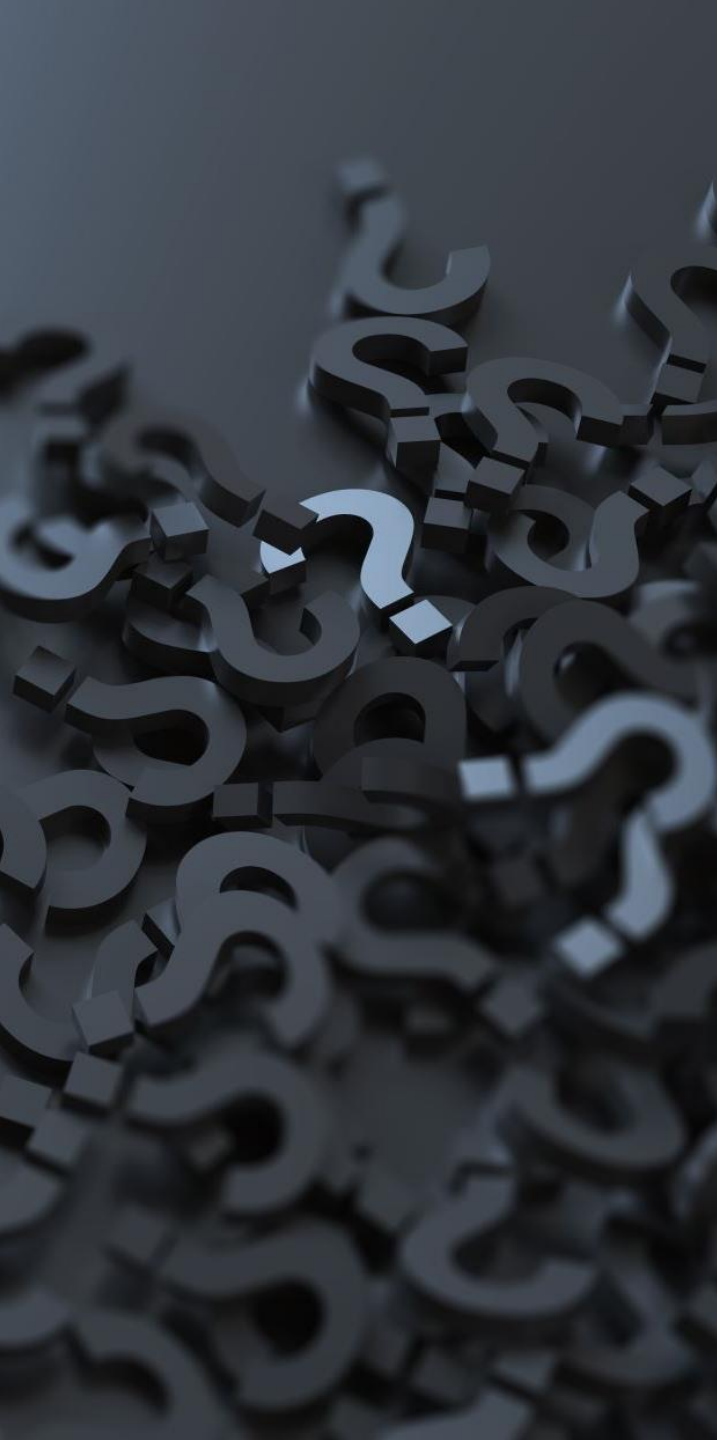
Technique and Algorithm



Why shift from prevention?

- Deadlock-prevention algorithms prevent deadlocks by limiting how requests can be made.
- The limits ensure that at least one of the necessary conditions for deadlock cannot occur.
- Possible side effects of preventing deadlocks by this method are:
 - Low device utilization, and
 - Reduced system throughput.





How to avoid deadlocks?

- Require additional information about how resources are to be requested.
- With knowledge of the complete sequence of requests and releases for each process, the system can decide for each request, whether the process should wait in order to avoid a possible future deadlock.
- For each request, the system considers:
 - The resources currently available,
 - The resources currently allocated to each process, and
 - The future requests and releases of each process.





Strategy of avoidance

- The various algorithms that use this approach differ in the amount and type of information required.
- The simplest and most useful model requires that each process declare the maximum number of resources of each type that it may need.
- Given this a priori information, it is possible to construct an algorithm that ensures that the system will never enter a deadlocked state.
- A deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that a circular-wait condition can never exist.
- The resource allocation state is defined by the number of available and allocated resources and the maximum demands of the processes.



Safe State



Definition

- A state is safe if the system can allocate resources to each process (up to its maximum) in some order and still avoid a deadlock.
- More formally, a system is in a safe state only if there exists a safe sequence.
- A sequence of processes $\langle P_1, P_2, \dots, P_n \rangle$ is a safe sequence for the current allocation state if, for each P_i , the resource requests that P_i can still make can be satisfied by the currently available resources plus the resources held by all P_j , with $j < i$.



Definition

- If the resources that P_i needs are not immediately available, then P_i can wait until all P_j have finished.
- Then, P_i can obtain all its needed resources, complete its designated task, return its allocated resources, and terminate.
- When P_i terminates, P_{i+1} can obtain its needed resources, and so on.
- If no such sequence exists, then the system state is said to be unsafe.



Safe state and deadlock

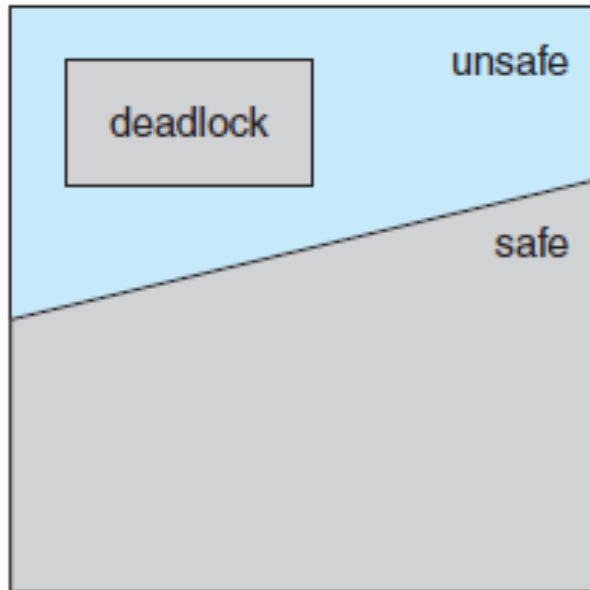


Figure 7.6 Safe, unsafe, and deadlocked state spaces.

- A safe state is not a deadlocked state. A deadlocked state is an unsafe state.
- Not all unsafe states are deadlocks. However, an unsafe state may lead to a deadlock.
- As long as the state is safe, the operating system can avoid unsafe (and deadlocked) states.
- In an unsafe state, the operating system cannot prevent processes from requesting resources in such a way that a deadlock occurs. The behavior of the processes controls unsafe states.



Is this state safe?

	<u>Maximum Needs</u>	<u>Current Needs</u>
P_0	10	5
P_1	4	2
P_2	9	2

-
- A system has 12 magnetic tape drives and three processes: P_0 , P_1 , and P_2 .
 - P_0 requires 10 tape drives, P_1 needs 4 tape drives, and P_2 needs 9 tape drives.
 - At time t_0 , P_0 , P_1 and P_2 hold 5, 2 and 2 tape drives respectively, while 3 are free.
 - Suppose that at t_0 ,
 1. We give 2 tape drives to P_1 . P_1 returns 4 tape drives, so 5 are free now.
 2. We can give all of them to P_0 . P_0 returns 10 tape drives, so 10 are free now.
 3. We can give 7 tape drives to P_2 . P_2 returns 9, and 12 are free now.
 - The safe sequence is $\langle P_1, P_0, P_2 \rangle$. We conclude that the state at time t_0 is safe.
 - Note that we have not given any resources to any process, nor has anyone made a request. We have only computed if a safe sequence exists. Thankfully, it does.



Can a safe state lead to an unsafe state?

	<u>Maximum Needs</u>	<u>Current Needs</u>
P_0	10	5
P_1	4	2
P_2	9	3

-
- Suppose that, at time t_1 , process P_2 requests and is allocated one more tape drive. At t_1 , P_0 , P_1 and P_2 hold 5, 2 and 3 tape drives respectively, while 2 are free. Can you compute a safe sequence for t_1 ?
 - Giving the two free drives to P_1 will satisfy all its needs. P_1 returns 4 tape drives. Now 4 are free. These 4 free tape drives will not be able to satisfy any other process, and P_0 and P_2 stay blocked. This is a deadlock.
 - The system is no longer in a safe state. Our mistake was in granting 1 more tape drive to P_2 . If we had made P_2 wait until either of the other processes had finished and released its resources, we could have avoided the deadlock.
 - A system can go from a safe state to an unsafe state.



Deadlock avoidance algorithm

- Avoidance algorithms that ensure that the system will never deadlock are algorithms that simply ensure that the system will always remain in a safe state.
- Initially, the system is in a safe state.
- Whenever a process requests a resource that is currently available, the system must decide whether the resource can be allocated immediately or whether the process must wait. dynamic decision
- The request is granted only if the allocation leaves the system in a safe state.
- In this scheme, if a process requests a resource that is currently available, it may still have to wait. Thus, resource utilization may be lower than it would otherwise be.



Deadlock avoidance algorithms

- We shall look at two deadlock avoidance algorithms:
 - The Resource-Allocation-Graph (RAG) algorithm, and
 - The banker's algorithm.



Resource- Allocation- Graph Algorithm



Limitations and edges

If we have a system with only one instance of each resource type, we can use the Resource-Allocation-Graph (RAG) algorithm for deadlock avoidance.

A directed edge $P_i \rightarrow R_j$ is called a request edge. It signifies that P_i has requested an instance of R_j and is currently waiting for it.

A directed edge $R_j \rightarrow P_i$ is called an assignment edge. It signifies that P_i holds an instance of R_j .

A directed dashed edge $P_i \rightarrow R_j$ is called a claim edge. It signifies that P_i may request an instance of R_j somewhere in the future.



Rules

When a process makes a request, its relevant claim edge converts to a request edge.

When the request is granted, the relevant request edge converts to an assignment edge.

When a process releases a resource after use, the assignment edge converts to a claim edge. This is because the process may repeatedly request the same resource.

The resources must be claimed a priori in the system. That is, in the beginning, we only have claim edges.

We can relax this condition by allowing a claim edge to be added to P_i in the graph only if all the edges associated with P_i are claim edges.

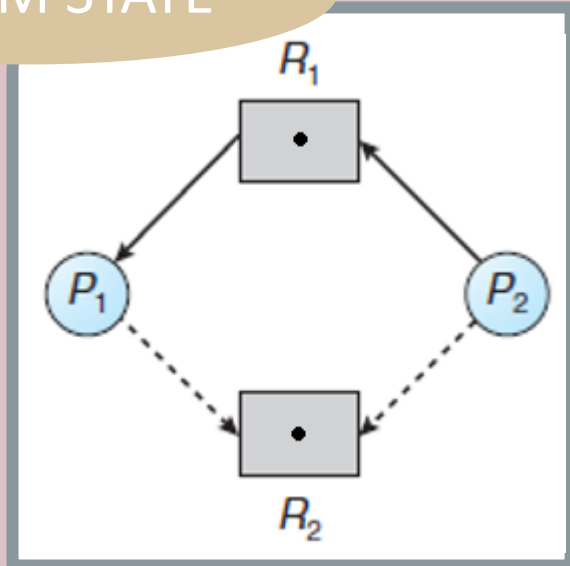


Evaluation of safe state

- Suppose P_i requests resource R_j , by converting its claim edge to a request edge.
- The request can be granted only if converting the request edge $P_i \rightarrow R_j$ to an assignment edge $R_j \rightarrow P_i$ does not result in the formation of a cycle in the RAG.
- We check for safety by using a cycle-detection algorithm. Such an algorithm requires an order of n^2 operations, where n is the number of processes in the system.
- If no cycle exists, then the system will be in a safe state after grant of request. The request is granted, and RAG changes accordingly.
- If a cycle is found, then the grant of this request will put the system in an unsafe state. The initial claim edge is restored from P_i to R_j . P_i is denied the request at this time and it waits.



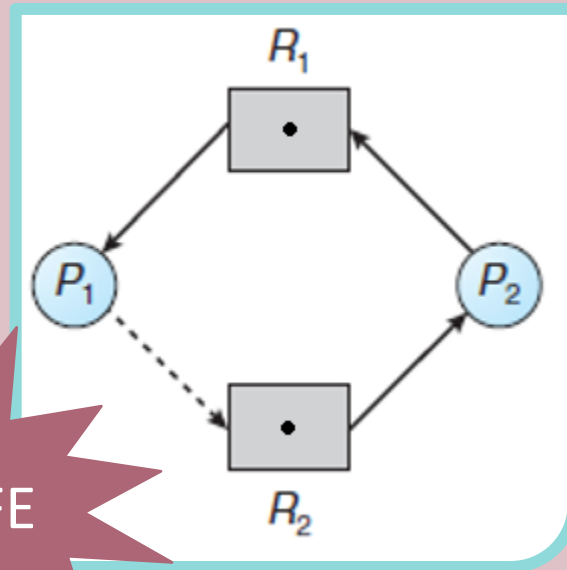
SYSTEM STATE



Example

- Suppose that P_2 requests R_2 .
- Although R_2 is currently free, we cannot allocate it to P_2 , since this action will create a cycle in the graph.
- If P_1 requests R_2 , and P_2 requests R_1 , then a deadlock will occur.

UNSAFE



Banker's Algorithm



About the algorithm...

- The resource-allocation-graph algorithm is not applicable to a system with multiple instances of each resource type. banker's algorithm is
- The deadlock avoidance algorithm that is applicable to such a system is commonly known as the banker's algorithm.
- The name was chosen because the algorithm could be used in a banking system to ensure that the bank never allocated its available cash in such a way that it could no longer satisfy the needs of all its customers.





This Photo by Unknown Author is licensed under [CC BY-SA](#)

Rules

-
- When a new process enters the system, it must declare the maximum number of instances of each resource type that it may need.
 - This number may not exceed the total number of resources in the system.
 - When a user requests a set of resources, the system must determine whether the allocation of these resources will leave the system in a safe state.
 - If it will, the resources are allocated; otherwise, the process must wait until some other process releases enough resources.



Data structures

- In our system, n is the number of processes and m is the number of resource types.
- In the banker's algorithm, these data structures encode the state of the system:
 - **Available**, a vector of order $1 \times m$.
 - **Max**, a matrix of order $n \times m$.
 - **Allocation**, a matrix of order $n \times m$.
 - **Need**, a matrix of order $n \times m$.



Elaboration of the data structures

- **Available** indicates the number of free resources of each type. If $\text{Available}[j] = k$, then k instances of R_j are available.
- **Max** defines the maximum demand of each process. If $\text{Max}[i][j] = k$, then P_i may request at most k instances of R_j .
- **Allocation** defines the number of resources of each type currently allocated to each process. If $\text{Allocation}[i][j] = k$, then P_i currently holds k instances of R_j .
- **Need** indicates the remaining resource needs of each process. If $\text{Need}[i][j] = k$, then P_i may need k more instances of R_j to complete its task. Note that:

$$\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$$





The $\leq$ relationship between vectors

- Let X and Y be vectors of length m .
- We say that $X \leq Y$ if and only if $X[i] \leq Y[i]$ for all $i = 1, 2, \dots, m$.
- If $X = (1, 7, 3, 2)$ and $Y = (0, 3, 2, 1)$, then $Y \leq X$.
- If $X = (1, 7, 3, 2)$ and $Y = (4, 8, 4, 2)$, then $X \leq Y$.

Pause the video!

Write your answer in a word file.

Save it as

<my_roll_no>_lecture_25. If your roll no is 500, the file will be named 500_lecture_25. Do not add <> signs to the filename.

Task# 1: $X = (1, 7, 3, 2)$ and $Y = (0, 3, 3, 1)$. Does any relationship hold between them?

$Y \leq X$ SINCE ALL VALUES OF X HAVE ATLEAST ONE VALUE IN Y THAT IS LESS
 $0 < 1, 3 < 7, 1 < 2$

Task# 2: $X = (1, 7, 3, 2)$ and $Y = (0, 8, 3, 1)$. Does any relationship hold between them?

NO RELATIONSHIP CAN BE ESTABLISHED



Working with rows of a larger matrix

- We can treat each row in a matrix as a row vector.
- We refer to the i^{th} row of Allocation and Need matrices as Allocation_i and Need_i .
- The vector Allocation_i specifies the resources currently allocated to P_i .
- The vector Need_i specifies the additional resources that P_i may still request to complete its task.





Safety algorithm:

For evaluation of safe state

1. Let **Work** and **Finish** be vectors of length **m** and **n**, respectively.
Initialize **Work = Available** and **Finish[i] = False** for $i = 0, 1, \dots, n - 1$.
2. Find an index i such that both **Finish[i] == False** and $\text{Need}_i \leq \text{Work}$. If no such i exists, go to step 4.
3. **Work = Work + Allocation_i**, **Finish[i] = True**, and go to step 2.
4. If **Finish[i] == True** for all i , then the system is in a safe state.
 - This algorithm may require an order of **$m \times n^2$** operations to determine whether a state is safe.

Resource subscript same as process subscript





Resource-request algorithm:

To determine if a request can be granted safely

- Let Request_i be the request vector for process P_i . If $\text{Request}_i[j] = k$, then P_i wants k instances of R_j . For Request_i , these actions are taken:
 1. If $\text{Request}_i \leq \text{Need}_i$, go to step 2. Otherwise, raise an error condition, since the process has exceeded its maximum claim.
 2. If $\text{Request}_i \leq \text{Available}$, go to step 3. Otherwise, P_i must wait, since the resources are not available. Resource subscript same as process subscript
 3. Have the system pretend to allocate the requested resources to P_i :
$$\text{Available} = \text{Available} - \text{Request}_i, \text{Allocation}_i = \text{Allocation}_i + \text{Request}_i, \text{Need}_i = \text{Need}_i - \text{Request}_i$$
- If the resulting state is safe, Request_i is granted to P_i . However, if the new state is unsafe, then P_i must wait for Request_i , and the old state is restored.



Is this system in safe state?

Process	Allocation			Max		
	A	B	C	A	B	C
P0	0	1	0	7	5	3
P1	2	0	0	3	2	2
P2	3	0	2	9	0	2
P3	2	1	1	2	2	2
P4	0	0	2	4	3	3

Available		
A	B	C
3	3	2

- Consider a system with 5 processes P_0 through P_4 and 3 resource types A, B, and C.
- A has 10 instances, B has 5 instances, and C has 7 instances.

allocation + available

max tells w many instances were asked by each process of a,b and c at the time of enterance



Is this system in safe state?

Process	Allocation			Max			Need		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3
P1	2	0	0	3	2	2	1	2	2
P2	3	0	2	9	0	2	6	0	0
P3	2	1	1	2	2	2	0	1	1
P4	0	0	2	4	3	3	4	3	1

- Compute for each P_i , $Need_i$:
 $Need_i = Max_i - Allocation_i$
- Now we do not need Max.
- We will be working with Allocation and Need.

0-1-0 minus from 7-5-3 yields 7-4-3 so maximum table is not needed further



Is this state safe?

Process		Allocation			Need		
Finish		A	B	C	A	B	C
P0	F	0	1	0	7	4	3
P1	F	2	0	0	1	2	2
P2	F	3	0	2	6	0	0
P3	F	2	1	1	0	1	1
P4	F	0	0	2	4	3	1

Available		
A	B	C
3	3	2
Work		
A	B	C
3	3	2

- Run the safety algorithm.
- Set all finish to False.
- Set Work = Available.

we compare with need, but allocation is added to work when matched the criteria



Is this state safe?

Process		Allocation			Need		
Finish		A	B	C	A	B	C
P0	F	0	1	0	7	4	3
P1	T	2	0	0	1	2	2
P2	F	3	0	2	6	0	0
P3	F	2	1	1	0	1	1
P4	F	0	0	2	4	3	1

Work		
A	B	C
5	3	2

- Find an index i such that both $\text{Finish}[i] == \text{False}$ and $\text{Need}_i \leq \text{Work}$.
- $\text{Need}_1 \leq \text{Work}!$
- $\text{Work} = \text{Work} + \text{Allocation}_1$
- $\text{Finish}[1] = \text{True}$

p1s allocation is added to the work because p1s resources were granted and are free now
p2 need not less than work but p3 is we run sequentially on each phase to process to check the criteria



Is this state safe?

Process		Allocation			Need		
Finish		A	B	C	A	B	C
P0	F	0	1	0	7	4	3
P1	T	2	0	0	1	2	2
P2	F	3	0	2	6	0	0
P3	T	2	1	1	0	1	1
P4	F	0	0	2	4	3	1

Work		
A	B	C
7	4	3

- Find an index i such that both $\text{Finish}[i] == \text{False}$ and $\text{Need}_i \leq \text{Work}$.
- $\text{Need}_3 \leq \text{Work}$!
- $\text{Work} = \text{Work} + \text{Allocation}_3$
- $\text{Finish}[3] = \text{True}$



Is this state safe?

Process		Allocation			Need		
Finish		A	B	C	A	B	C
P0	F	0	1	0	7	4	3
P1	T	2	0	0	1	2	2
P2	F	3	0	2	6	0	0
P3	T	2	1	1	0	1	1
P4	T	0	0	2	4	3	1

Work		
A	B	C
7	4	5

- Find an index i such that both $\text{Finish}[i] == \text{False}$ and $\text{Need}_i \leq \text{Work}$.
- $\text{Need}_4 \leq \text{Work}!$
- $\text{Work} = \text{Work} + \text{Allocation}_4$
- $\text{Finish}[4] = \text{True}$

wrap around and start from p0



Is this state safe?

Process		Allocation			Need		
Finish		A	B	C	A	B	C
P0	T	0	1	0	7	4	3
P1	T	2	0	0	1	2	2
P2	F	3	0	2	6	0	0
P3	T	2	1	1	0	1	1
P4	T	0	0	2	4	3	1

Work		
A	B	C
7	5	5

- Find an index i such that both $\text{Finish}[i] == \text{False}$ and $\text{Need}_i \leq \text{Work}$.
- $\text{Need}_0 \leq \text{Work}!$
- $\text{Work} = \text{Work} + \text{Allocation}_0$
- $\text{Finish}[0] = \text{True}$

p1 finish is T so we skip it



Is this state safe?

Process		Allocation			Need		
Finish		A	B	C	A	B	C
P0	T	0	1	0	7	4	3
P1	T	2	0	0	1	2	2
P2	T	3	0	2	6	0	0
P3	T	2	1	1	0	1	1
P4	T	0	0	2	4	3	1

Work		
A	B	C
10	5	7

- Find an index i such that both $\text{Finish}[i] == \text{False}$ and $\text{Need}_i \leq \text{Work}$.
- $\text{Need}_2 \leq \text{Work}!$
- $\text{Work} = \text{Work} + \text{Allocation}_2$
- $\text{Finish}[2] = \text{True}$



Is this state safe?

- Finish[i] == True for all i
- So the state is safe.
- The safe sequence is:

$\langle P_1, P_3, P_4, P_0, P_2 \rangle$

pehly kiska, phir kiska

Process		Allocation			Need		
Finish		A	B	C	A	B	C
P0	T	0	1	0	7	4	3
P1	T	2	0	0	1	2	2
P2	T	3	0	2	6	0	0
P3	T	2	1	1	0	1	1
P4	T	0	0	2	4	3	1

Work		
A	B	C
10	5	7



Lessons from the lecture

We understood why deadlock prevention is not a good option.

We realized how deadlock avoidance better utilizes system resources.

We listed the algorithms for deadlock avoidance.

We explored the Resource-Allocation-Graph algorithm.

We explored the banker's algorithm.



Things to do

